APPLICATIONS WEB DISTRIBUÉES

MediConnect

Plateforme intelligente de coordination des services médicaux

Validation 1 · Conception microservices



ÉQUIPE

Rawen Aouina · Sarra Mejbri · Bochra Damak
Rania Ben Haj Hmida · Bechir Belgaied · Yassine Gasmi

PLAN

01 **Sujet et fonctionnalités**
problématique, cas d'utilisation, périmètre

02 **Découpage en microservices**
un service par capacité métier

03 **Architecture, données et sécurité**
schéma global, une base adaptée à chaque service, Keycloak

04 **Technologies**
stack choisie, organisée par couche

05 **Organisation, planning et risques**
Git, Jira

06 **Les microservices en détail**
MS1 à MS7 : rôle, conception, interactions

Contexte, problématique et objectifs

Contexte

Transformation numérique
Le numérique facilite l'accès aux services médicaux.

Services dispersés
Répartis entre établissements et plateformes.

Coordination limitée
Échanges difficiles entre acteurs de santé.

PROBLÉMATIQUE

Comment centraliser les différents services médicaux dans une seule plateforme afin de faciliter leur accès et améliorer la coordination entre les acteurs de santé ?

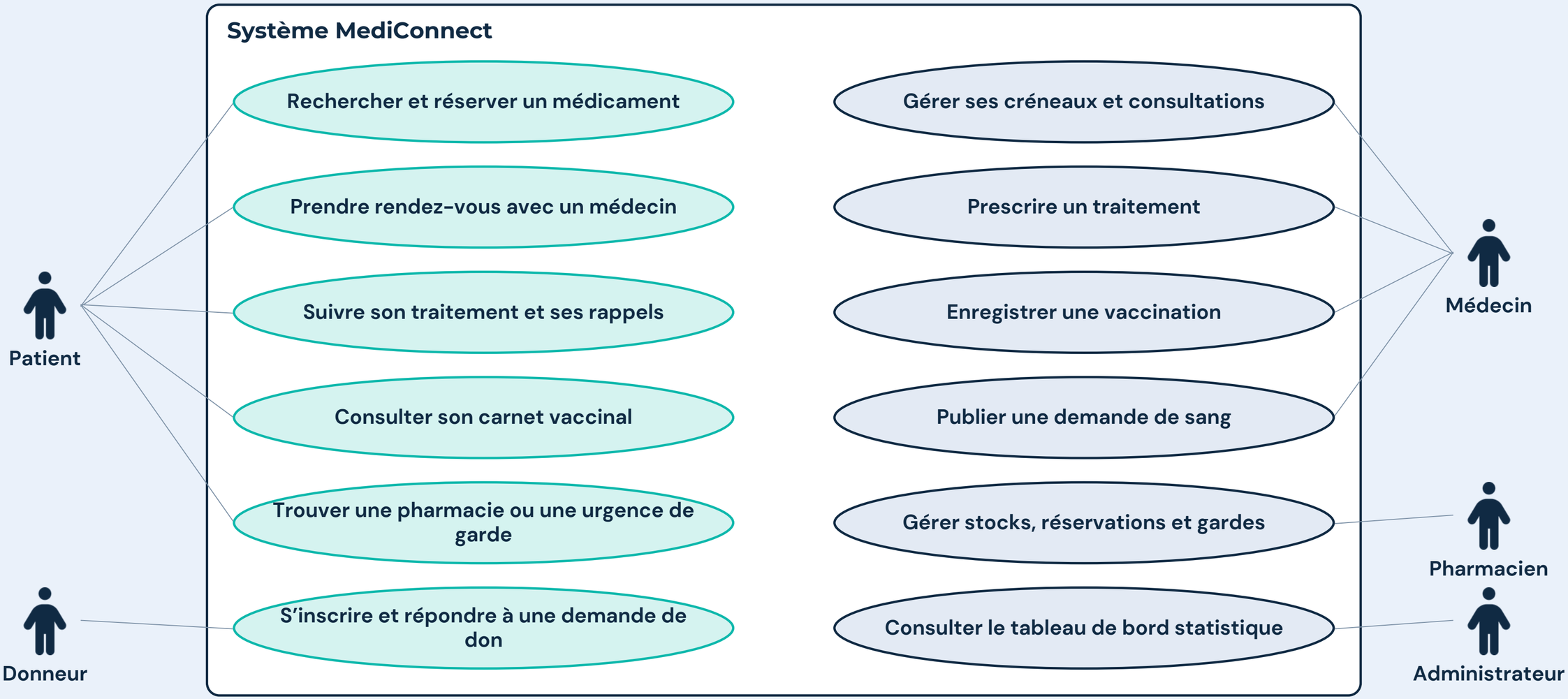
NOTRE RÉPONSE

MediConnect centralise plusieurs services médicaux dans une seule plateforme.

- **Centraliser les services médicaux**
- **Faciliter l'accès aux médicaments**
- **Simplifier les rendez-vous médicaux**

Cas d’utilisation général

Cinq acteurs, douze cas d’utilisation principaux, tous accessibles après authentification Keycloak.



Principales fonctionnalités et périmètre

Inclus dans le projet

1 par service

- Réserver un médicament en pharmacie
- Publier et suivre les demandes de don
- Prendre rendez-vous avec un médecin
- Recevoir les rappels de traitement
- Trouver une pharmacie ou une urgence de garde
- Tenir son carnet de vaccination
- Suivre les indicateurs de la plateforme

Hors périmètre

- Diagnostic médical automatisé
- Prescription médicale autonome
- Paiement en ligne
- Téléconsultation (non prévue à ce stade)

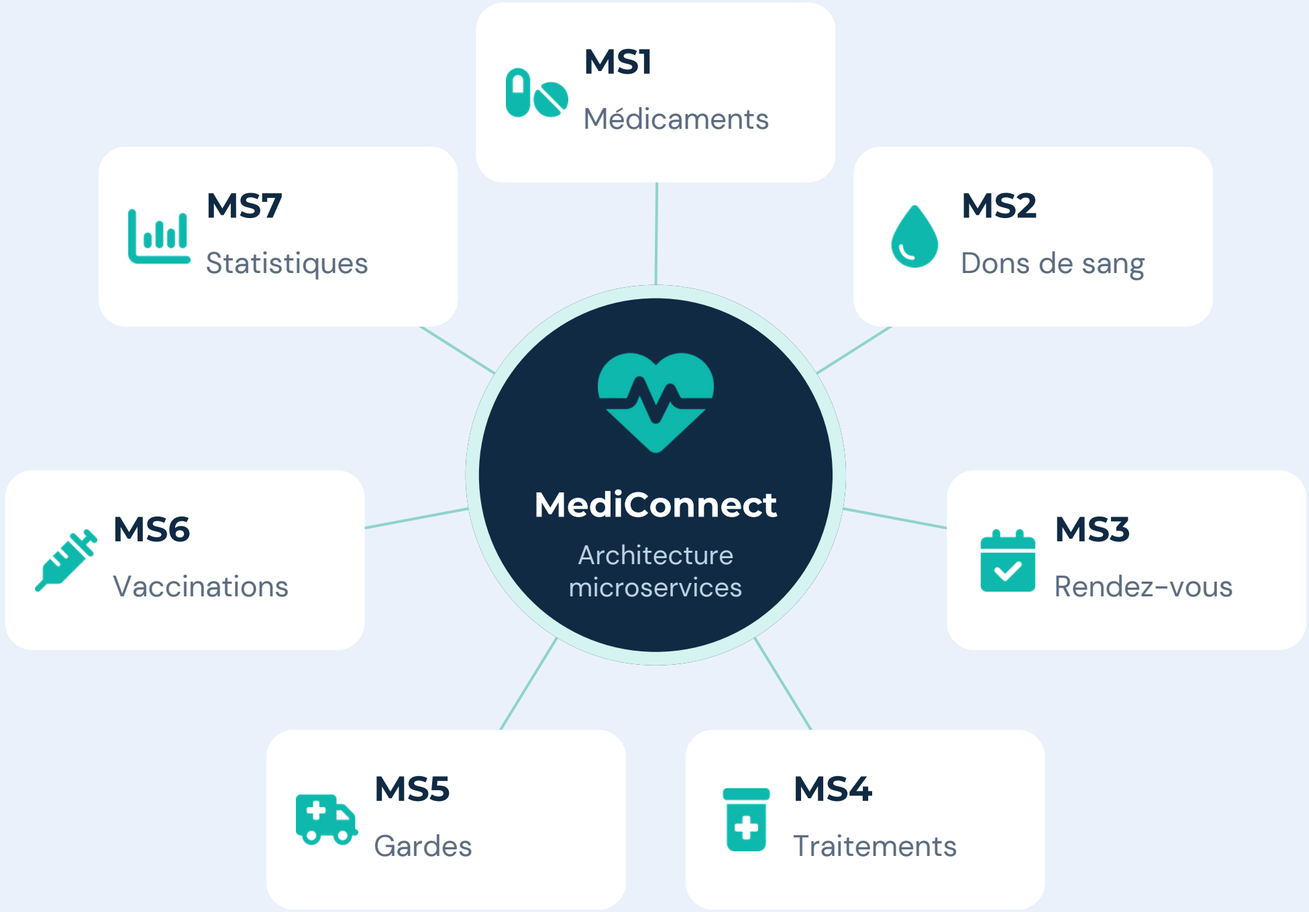
MediConnect facilite l'accès et la coordination des services ; les décisions médicales restent du ressort des professionnels de santé.

Conception et découpage en microservices

Un microservice par capacité métier (contexte borné), sans recouvrement ; MS7 est le service commun.

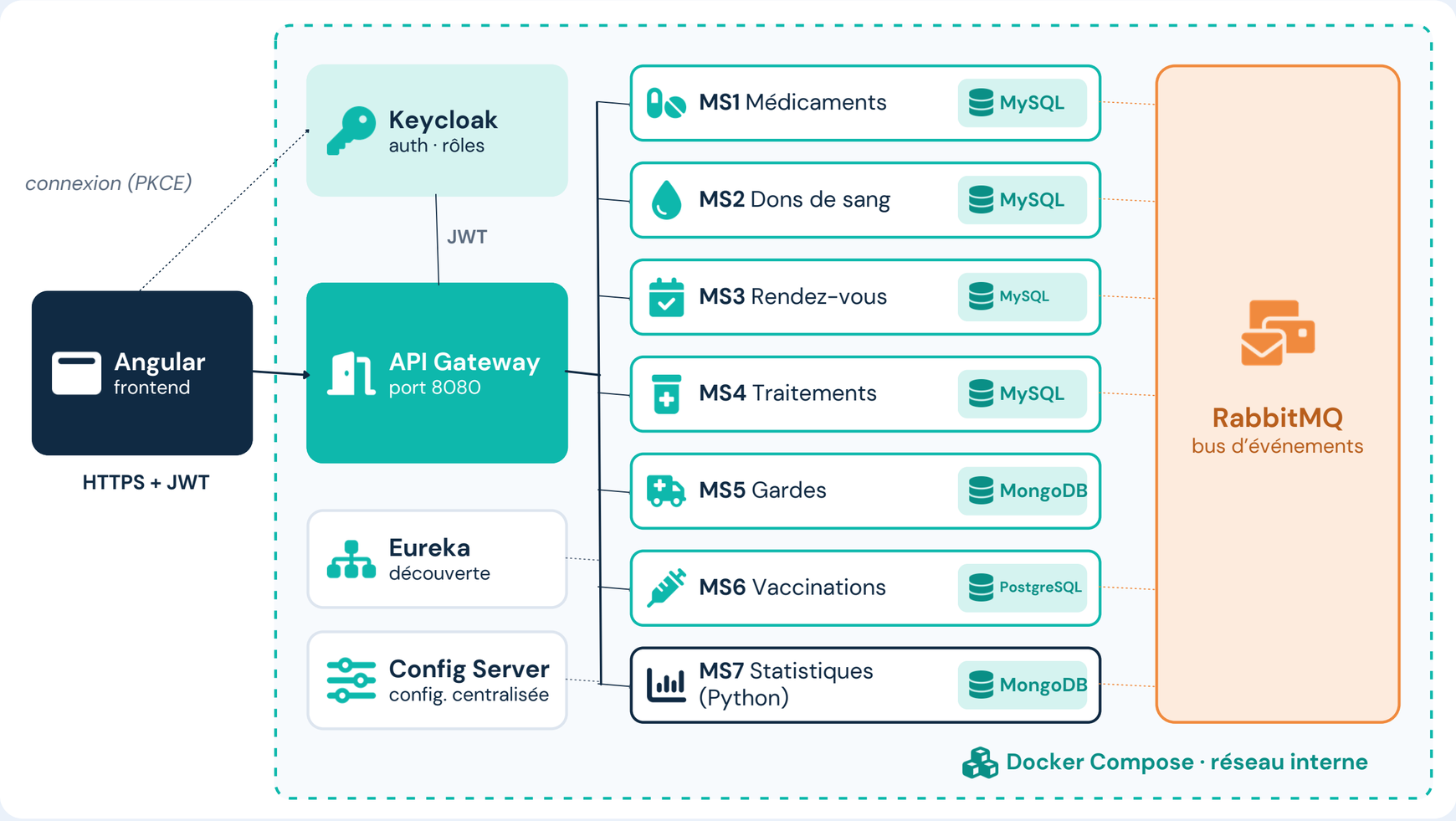
Les 7 microservices

- **MS1** Médicaments & pharmacies
- **MS2** Dons de sang
- **MS3** Rendez-vous médicaux
- **MS4** Traitements & rappels
- **MS5** Gardes & urgences
- **MS6** Vaccinations
- **MS7** Analyse & statistiques



Architecture cible

Synchrone (OpenFeign) quand la réponse est nécessaire, asynchrone (RabbitMQ) sinon · déployé avec Docker Compose.



BRIQUES CLÉS

API Gateway
Point d'entrée unique · 8080

Eureka
Découverte des services

Config Server
Configuration centralisée

Keycloak
Authentification et rôles

RabbitMQ
Événements asynchrones

Bases dédiées
MySQL · PostgreSQL · MongoDB

Docker Compose
Déploiement des conteneurs

Gestion des données et sécurité

Chaque service possède ses données ; tout accès passe par la Gateway et Keycloak.

Une base adaptée à chaque service

MySQL / PostgreSQL : transactions ACID · MongoDB : documents et géolocalisation · H2 pour les tests. Aucune base partagée.

MS1 · MySQL
stock et réservations en transaction

MS2 · MySQL
donneurs, dons et demandes liés

MS3 · MySQL
aucun chevauchement de créneaux

MS4 · MySQL
traitement → prescriptions → prises

MS5 · MongoDB
recherche de la plus proche

MS6 · PostgreSQL
historique vaccinal fiable

MS7 · MongoDB
événements JSON variés

Authentification et autorisation

1. **Keycloak** Angular redirige vers Keycloak (Authorization Code + PKCE), qui émet le JWT
2. **Angular** ajoute « Authorization: Bearer <JWT> » à chaque requête
3. **API Gateway** vérifie la signature et l'expiration du jeton (clés publiques Keycloak)
4. **Microservice** revérifie le JWT et contrôle le rôle (@PreAuthorize)

Seule la Gateway est exposée : services et bases de données restent dans le réseau Docker interne.

RÔLES — Patient · Médecin · Pharmacien · Donneur · Admin

Technologies utilisées

La stack imposée par le module, complétée par un service commun en Python, organisée par couche.

Présentation	**Angular** Interfaces modulaires par domaine	
Accès & sécurité	**Spring Cloud Gateway** Routage, filtres, CORS	**Keycloak + JWT** SSO, rôles et contrôle d'accès
Services métier	**Spring Boot** MS1 à MS6 · REST + Swagger	**Python · FastAPI** MS7 commun · technologie avancée
Infrastructure	**Eureka** Découverte des services	**Config Server** Configuration centralisée **Docker Compose** Conteneurs reproductibles
Échanges & données	**RabbitMQ** Événements asynchrones	**MySQL · PostgreSQL · MongoDB** Une base adaptée par service

Organisation Git et gestion du projet

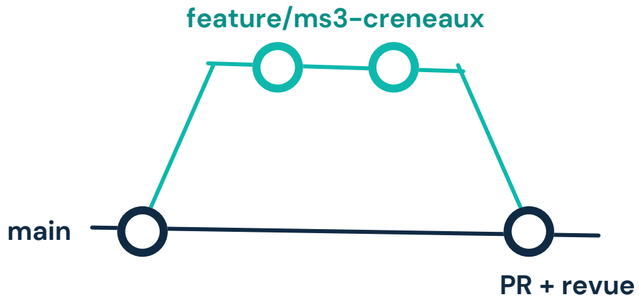
Dépôt public à la structure imposée, un projet par microservice, suivi des tâches dans Jira.

Dépôt Git · structure imposée

- **mediconnect-classe-groupe/**
  - **Backend/**
    - gateway/
    - eureka-server/ · config-server/
    - ms1-medicaments/ … ms6-vaccinations/
    - ms7-statistiques/ (Python)
  - **Frontend/ (Angular)**
  - **Documentation/**
    - Presentation/ · Diagrams/
    - dossier-technique.docx
  - docker-compose.yml
  - README.md

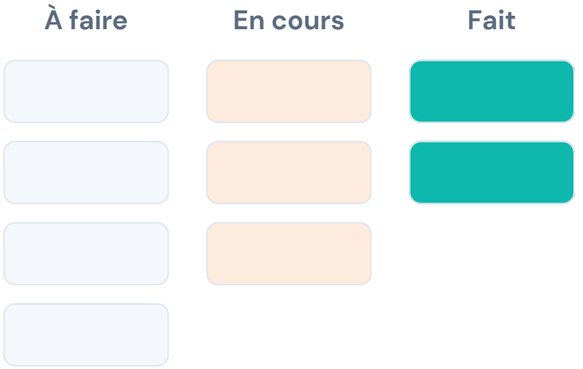
Stratégie de branches

- Branche main protégée
- Une branche par fonctionnalité
- Pull request + revue avant fusion



Gestion · Jira

- Un backlog par microservice
- Un tableau infrastructure
- Revue à chaque séance



Planning prévisionnel et risques

Une étape vérifiable à chaque séance, du squelette à la démonstration finale.

S4 — Aujourd'hui — S5 — S6 — S7 — S8 — S9 — Fin

Validation 1
Conception validée, projets des 7 services créés

Infrastructure
Eureka, Config Server, routage Gateway

Métier
Endpoints principaux, une base dédiée par service

Sécurité
Keycloak + JWT, appels OpenFeign

Événements
RabbitMQ, service MS7, écrans Angular

Intégration
Docker Compose, tests de bout en bout, README

Soutenance
Démonstration du parcours complet

PRINCIPAUX RISQUES ET PARADES

Contrats qui changent
→ Swagger figé dès S4

Panne d'un service appelé
→ Erreur claire, pas de blocage

Intégration tardive
→ Intégrer à chaque séance

Charge inégale
→ Suivi Jira par membre

MS1 — Médicaments et pharmacies

Spring Boot

Base MySQL dédiée

Rôle



Gérer le catalogue des médicaments, leur disponibilité en pharmacie et les réservations.

Fonctions clés

- Catalogue des médicaments
- Disponibilité par pharmacie
- Réservation de médicaments
- Gestion des stocks

Données clés

- Médicament
- Pharmacie
- Stock
- Réservation

Stockées dans la base MySQL de MS1

Objectif

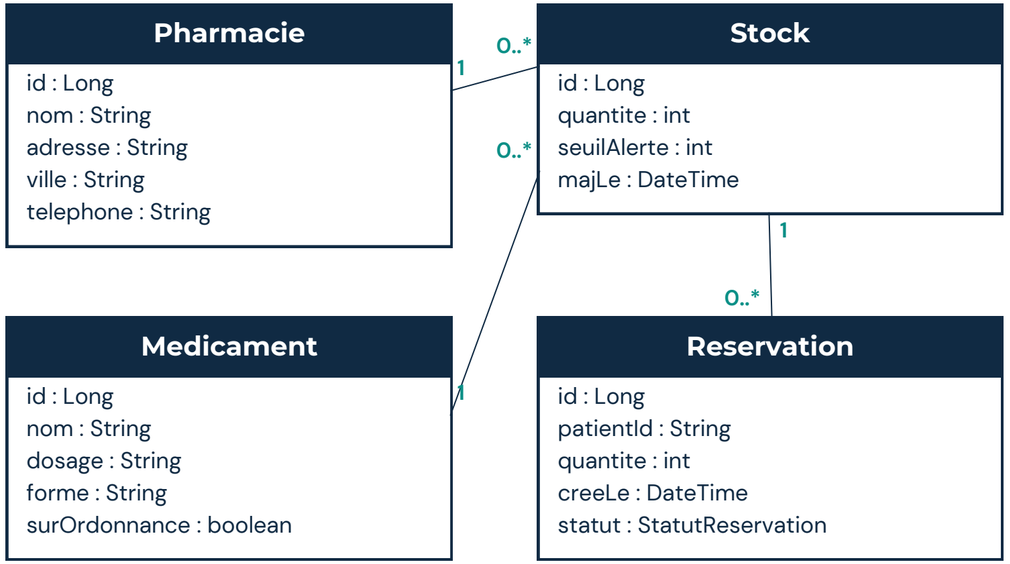
Faciliter l'accès aux médicaments.

MS1 — Conception : classes et API

Spring Boot

Base MySQL dédiée

Diagramme de classes



«enum» StatutReservation

EN_ATTENTE · CONFIRMEE · RETIREE · ANNULEE

Endpoints et codes

GET **/api/medications?nom=**
200 · 400

GET **/api/medications/{id}/availability**
200 · 404

POST **/api/reservations**
201 + Location · 400 · 409

PATCH **/api/reservations/{id}**
200 · 404 · 409

ÉVÉNEMENTS

Publie → MS7 : ReservationCreee

Appelé en synchrone par MS4 et MS5

Choix BD

MySQL : stock et réservation modifiés dans une même transaction (pas de sur-réservation).

MS1 — Interactions

Spring Boot

Base MySQL dédiée

Appelle



Aucun appel sortant

MS1 est le référentiel
des médicaments et des pharmacies

Appelé par · événements

MS4 l'appelle (synchrone)
nom et dosage d'un médicament

MS5 l'appelle (synchrone)
nom, adresse, téléphone d'une pharmacie

Vers MS7 : ReservationCreee
statistiques des réservations

En cas de panne

MS7 indisponible
l'événement reste dans RabbitMQ

Réservation enregistrée
même si MS7 est arrêté

Stock déjà réservé
refus clair (409)

Aucun blocage pour le patient

Pourquoi **Synchrone quand MS4 et MS5 ont besoin de la donnée tout de suite, asynchrone vers MS7.**

MS2 — Dons de sang

Spring Boot

Base MySQL dédiée

Rôle



Gérer les donneurs, les demandes et les dons de sang.

Fonctions clés

- Inscription des donneurs
- Gestion des groupes sanguins
- Demandes de sang
- Suivi des dons

Données clés

- Donneur
- Demande
- Don
- Groupe sanguin

Stockées dans la base MySQL de MS2

Objectif

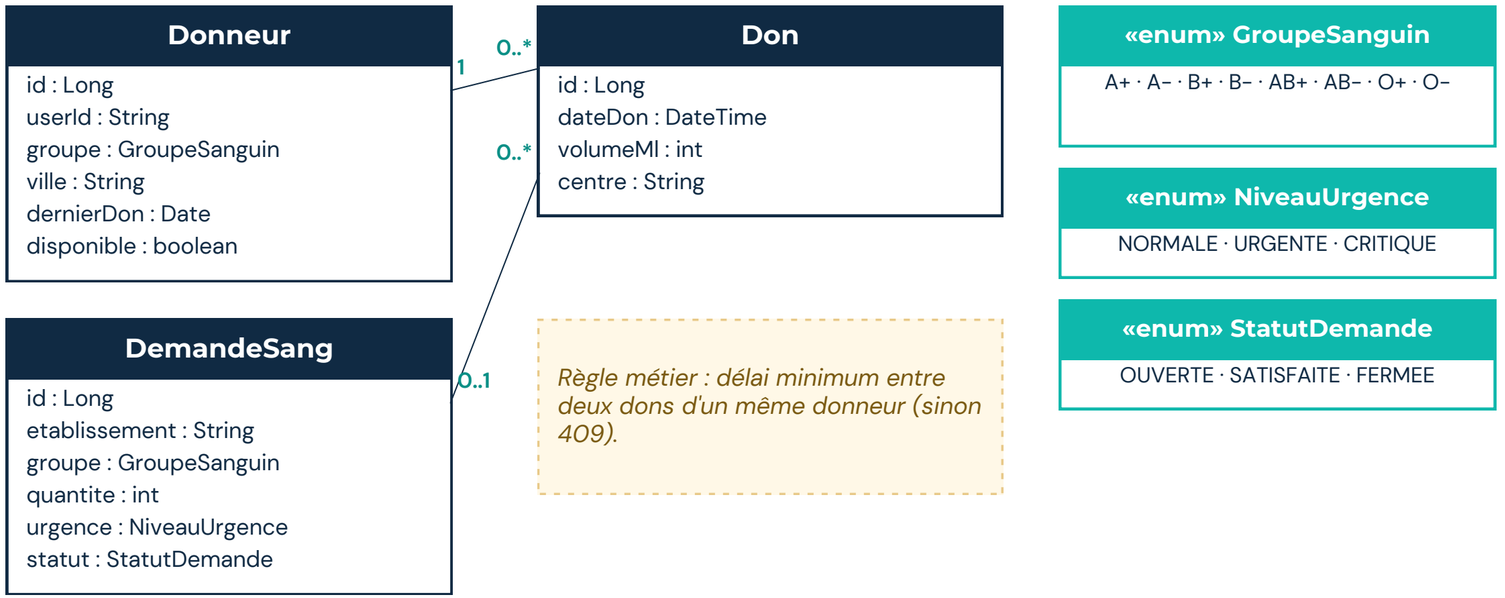
Faciliter la mise en relation pour les dons de sang.

MS2 — Conception : classes et API

Spring Boot

Base MySQL dédiée

Diagramme de classes



Endpoints et codes

POST `/api/donors`
201 + Location · 400 · 409

POST `/api/blood-requests`
201 + Location · 400

GET `/api/blood-requests?groupe=&ville=`
200 · 400

POST `/api/donations`
201 + Location · 400 · 409

ÉVÉNEMENTS

Publie → MS7 : DemandeSangPubliee, DonEnregistre

Aucun appel synchrone sortant

Choix BD

MySQL : relations donneur → dons ← demandes, contraintes d'intégrité.

MS2 — Interactions

Spring Boot

Base MySQL dédiée

Appelle

Aucun appel synchrone

service autonome :
ses propres données suffisent

Appelé par · événements

Angular via la Gateway
inscription, demandes, dons

Vers MS7 : DemandeSangPubliee
demandes par groupe sanguin

Vers MS7 : DonEnregistre
volume de dons collectés

En cas de panne

MS7 indisponible
événements gardés en file d'attente

RabbitMQ indisponible
événement republié plus tard

Don enregistré
normalement dans MS2

Aucun blocage pour le donneur

Pourquoi **Les statistiques n'ont pas besoin d'une réponse immédiate : communication asynchrone.**

MS3 — Rendez-vous médicaux

Spring Boot

MySQL dédiée

Rôle



Gérer les médecins, les créneaux et les rendez-vous des patients.

Fonctions clés

- Gestion des médecins
- Disponibilités
- Prise de rendez-vous
- Historique des consultations

Données clés

- Médecin
- Patient
- Créneau
- Rendez-vous

Stockées dans la base MySQL de MS3

Objectif

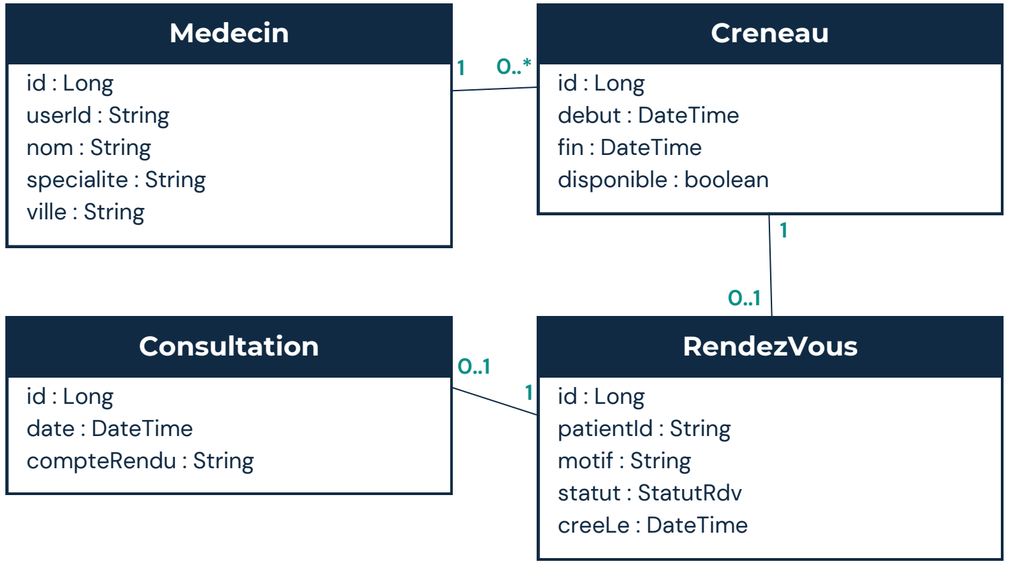
Simplifier l'organisation des rendez-vous médicaux.

MS3 — Conception : classes et API

Spring Boot

MySQL dédiée

Diagramme de classes



Endpoints et codes

GET **/api/doctors?specialite=&ville=**
200 · 400

GET **/api/doctors/{id}/slots**
200 · 404

POST **/api/appointments**
201 + Location · 400 · 409

PATCH **/api/appointments/{id}/cancel**
200 · 404 · 409

ÉVÉNEMENTS

Publie → MS4, MS7 : RendezVousReserve / Annule

Appelle MS4 en synchrone

Choix BD

PostgreSQL : contrainte d'unicité et vérification des créneaux horaires pour empêcher deux rendez-vous d'un même médecin de se chevaucher.

MS3 — Interactions

Spring Boot

MySQL dédiée

Appelle



MS4 · Traitements

Synchrone (OpenFeign)
traitements en cours du patient

Appelé par · événements

Angular via la Gateway
créneaux et réservations

Vers MS4 : RendezVousReserve
rappel la veille du rendez-vous

Vers MS7 : RendezVousReserve
statistiques des rendez-vous

Vers MS4, MS7 : RendezVousAnnule
rappel supprimé, statistiques à jour

En cas de panne

MS4 indisponible
consultation affichée sans traitements

Message clair
« traitements indisponibles »

Événements
gardés en file d'attente

La réservation n'est jamais bloquée

Pourquoi

Synchrone pour montrer les traitements au médecin, asynchrone pour les rappels et les statistiques.

MS4 — Traitements et rappels

Spring Boot

Base MySQL dédiée

Rôle



Suivre les traitements prescrits et rappeler les prises aux patients.

Fonctions clés

- Création des traitements
- Planification des prises
- Rappels
- Historique du suivi

Données clés

- Traitement
- Prescription
- Rappel
- Historique

Stockées dans la base MySQL de MS4

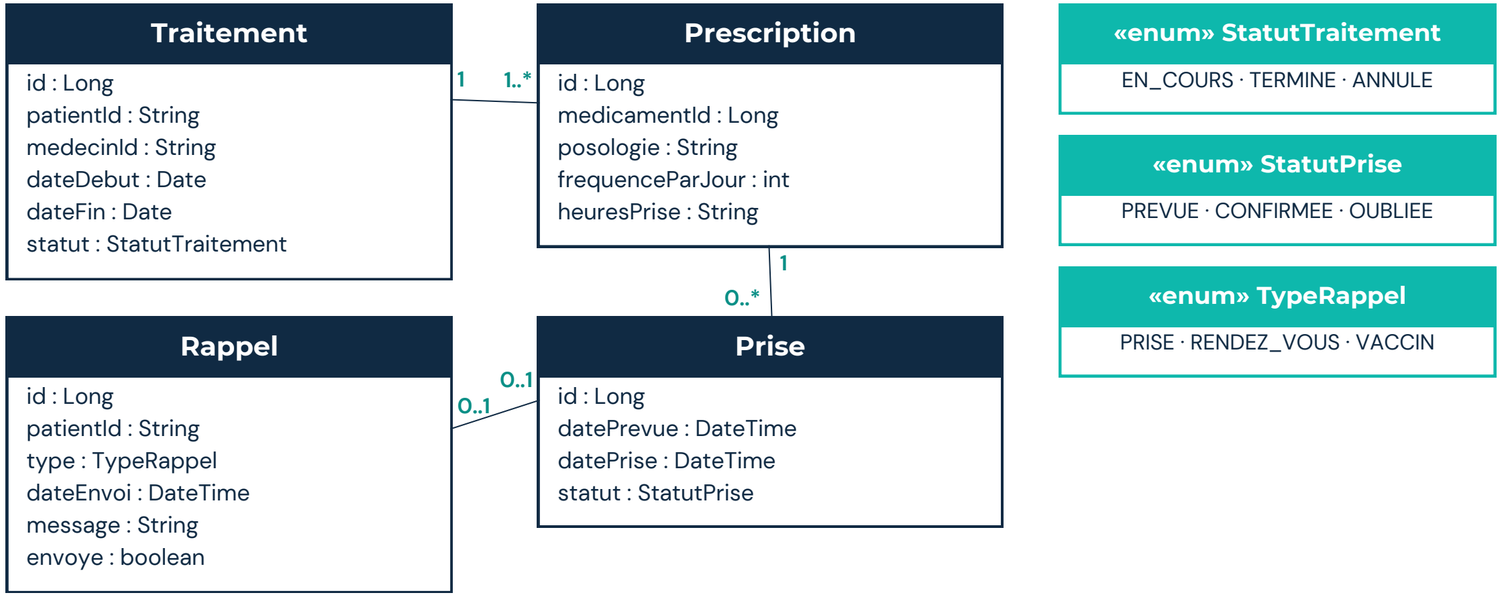
Objectif — **Améliorer le suivi des traitements.**

MS4 — Conception : classes et API

Spring Boot

Base MySQL dédiée

Diagramme de classes



«enum» StatutTraitement

EN_COURS · TERMINE · ANNULE

«enum» StatutPrise

PREVUE · CONFIRMEE · OUBLIEE

«enum» TypeRappel

PRISE · RENDEZ_VOUS · VACCIN

Endpoints et codes

POST **/api/treatments**
201 + Location · 400

GET **/api/treatments?patientId=**
200 · 400

GET **/api/treatments/{id}/intakes**
200 · 404

PATCH **/api/intakes/{id}**
200 · 404 · 409

ÉVÉNEMENTS

Publie → MS7 : PriseConfirmee, PriseOubliee

Consomme : RendezVousReserve, VaccinEnregistre

Choix BD

MySQL : données relationnelles (traitement → prescriptions → prises).

MS4 — Interactions

Spring Boot

Base MySQL dédiée

Appelle



MS1 · Médicaments

Synchrone (OpenFeign)
nom et dosage du médicament

Appelé par · événements

MS3 l'appelle (synchrone)
traitements en cours du patient

MS3 → RendezVousReserve
rappel la veille du rendez-vous

MS6 → VaccinEnregistre
rappel de la prochaine dose

Vers MS7 : PriseConfirmee, PriseOubliee

En cas de panne

MS1 indisponible
traitement créé avec l'identifiant seul

Nom affiché
« indisponible »

Événements
gardés en file d'attente

Aucun blocage pour le patient

Pourquoi

Synchrone si la réponse est requise tout de suite, asynchrone pour les rappels.

MS5 — Gardes et urgences

Spring Boot

MongoDB dédiée

Rôle



Indiquer les pharmacies et services de garde en cas d'urgence.

Fonctions clés

- Pharmacies de garde
- Services d'urgence
- Recherche par zone
- Informations pratiques

Données clés

- Pharmacie (réf. MS1)
- Garde
- Urgence
- Localisation

Stockées dans la base MongoDB de MS5

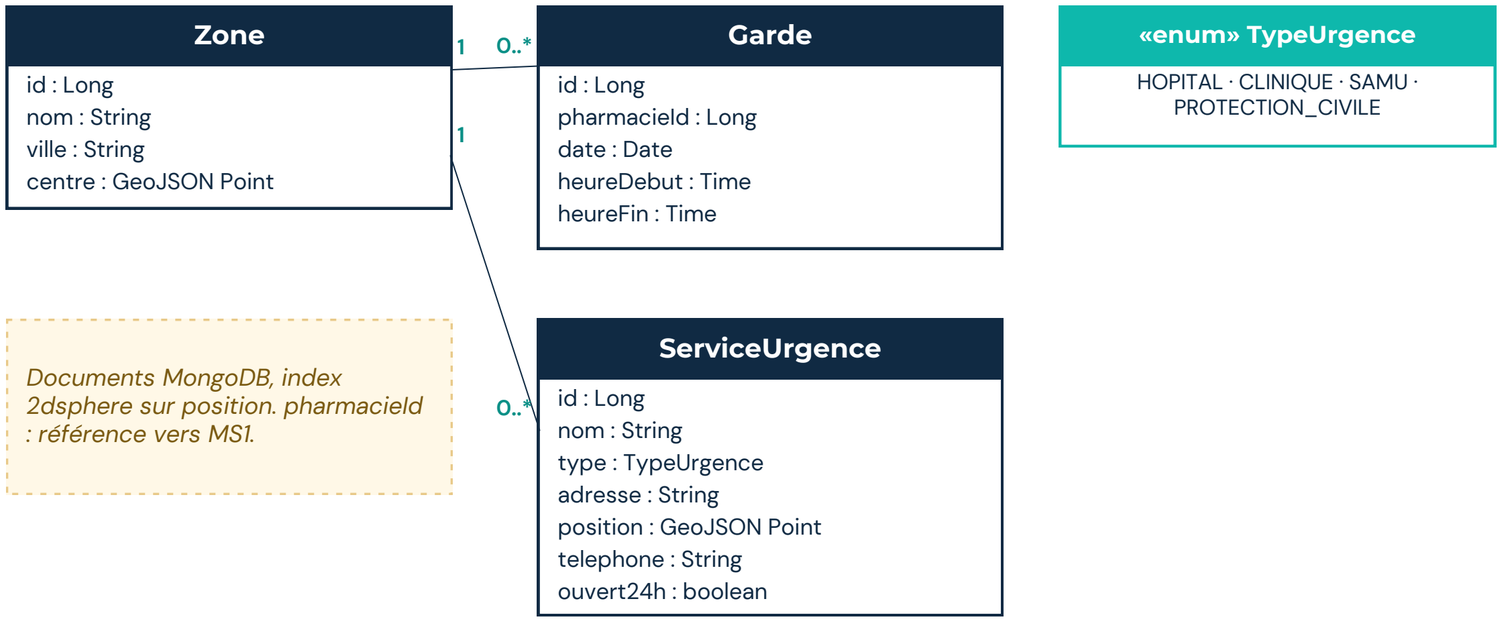
Objectif

Orienter rapidement le patient en situation d'urgence.

MS5 — Conception : classes et API

Spring Boot | MongoDB dédiée

Diagramme de classes



Endpoints et codes

GET	/api/on-call/pharmacies?zone=&date=	200 · 400
GET	/api/on-call/emergency-services	200 · 400
POST	/api/on-call/shifts	201 + Location · 400 · 409
DELETE	/api/on-call/shifts/{id}	204 · 404

ÉVÉNEMENTS

Aucun événement publié

Appelle MS1 en synchrone

Choix BD

MongoDB : index géospatial 2dsphere pour trouver la pharmacie ou l'urgence la plus proche ($near).

MS5 — Interactions

Spring Boot

MongoDB dédiée

Appelle



MS1 · Pharmacies

Synchrone (OpenFeign)
nom, adresse, téléphone

Appelé par · événements

Angular via la Gateway
recherche par zone et par date

Aucun événement publié
service de consultation

En cas de panne

MS1 indisponible
garde affichée (zone, horaires)

Nom de la pharmacie
« indisponible »

Urgences toujours affichées
données propres à MS5

Une urgence est toujours trouvée

Pourquoi **Synchrone : la pharmacie appartient à MS1, MS5 ne la recopie pas.**

MS6 — Vaccinations

Spring Boot

PostgreSQL dédiée

Rôle



Gérer le carnet vaccinal et les rappels de vaccination.

Fonctions clés

- Carnet numérique
- Enregistrement des doses
- Rappels (envoyés par MS4)
- Suivi des échéances

Données clés

- Vaccin
- Dose
- Calendrier
- Patient vacciné

Stockées dans la base PostgreSQL de MS6

Objectif

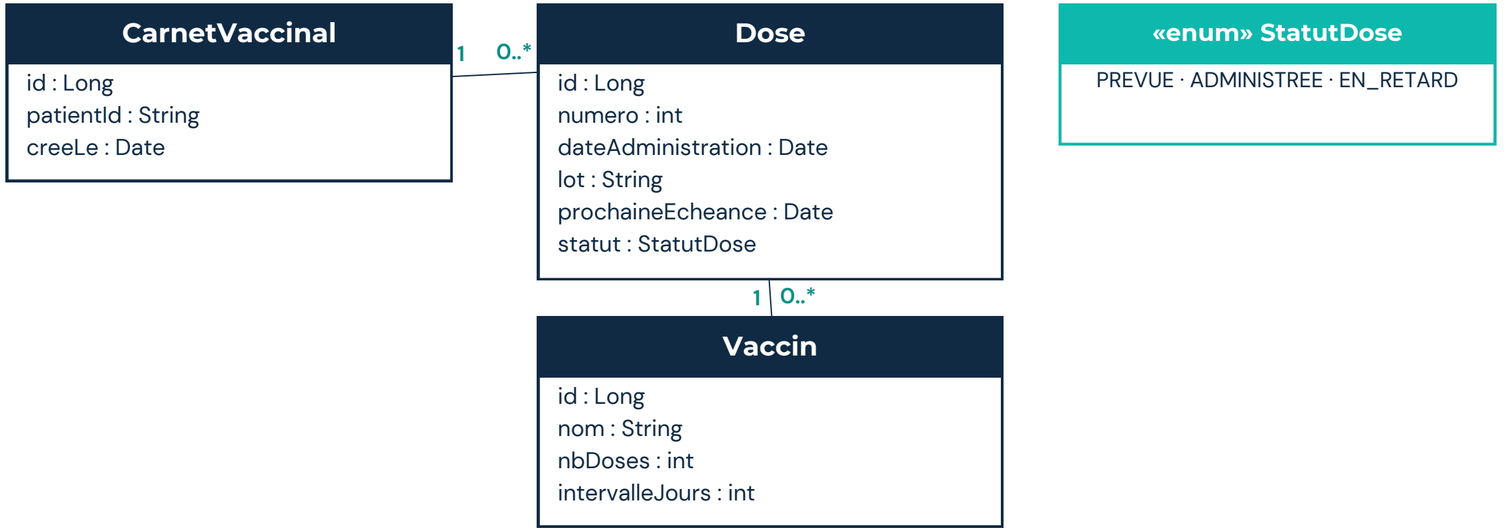
Assurer un meilleur suivi vaccinal.

MS6 — Conception : classes et API

Spring Boot

PostgreSQL dédiée

Diagramme de classes



Endpoints et codes

GET **/api/vaccines**
200

GET **/api/vaccination-records/{id}**
200 · 404

POST **/api/vaccinations**
201 + Location · 400 · 409

GET **/api/vaccination-records/{id}/due**
200 · 404

ÉVÉNEMENTS

Publie → MS4, MS7 : VaccinEnregistre

Aucun appel synchrone sortant

Choix BD

PostgreSQL : historique vaccinal fiable, contrainte d'unicité (carnet, vaccin, numéro de dose).

MS6 — Interactions

Spring Boot | PostgreSQL dédiée

Appelle

Aucun appel synchrone

le carnet vaccinal
est autonome

Appelé par · événements

Angular via la Gateway
carnet, doses, échéances

Vers MS4 : VaccinEnregistre
rappel de la prochaine dose

Vers MS7 : VaccinEnregistre
couverture vaccinale

En cas de panne

MS4 indisponible
rappel envoyé plus tard (file)

MS7 indisponible
statistiques mises à jour ensuite

Dose enregistrée
normalement dans MS6

Aucun blocage pour le patient

Pourquoi — **Asynchrone : le rappel et les statistiques n'ont pas besoin d'une réponse immédiate.**

MS7 — Analyse et statistiques (commun)

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Rôle



Centraliser les indicateurs et produire des statistiques globales.

Fonctions clés

- Tableaux de bord
- Agrégation des données
- Suivi des indicateurs
- Aide à la décision

Données clés

- Statistique
- Rapport
- Indicateur
- Synthèse

Stockées dans la base MongoDB de MS7

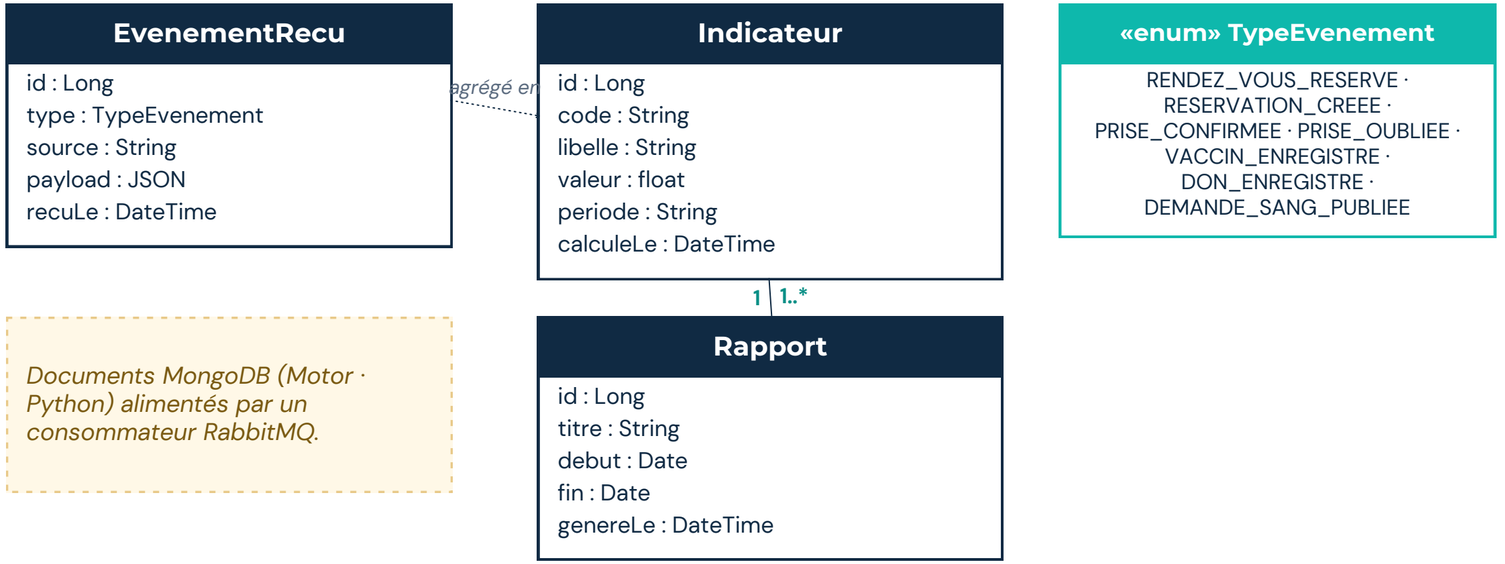
Objectif

Fournir une vue globale de la plateforme.

MS7 — Conception : classes et API

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Diagramme de classes



Endpoints et codes

GET	/api/analytics/dashboard	200 · 403
GET	/api/analytics/indicators?periode=	200 · 400
POST	/api/analytics/reports	201 + Location · 400
GET	/api/analytics/reports/{id}	200 · 404

ÉVÉNEMENTS

Consomme : événements de MS1, MS2, MS3, MS4, MS6

Aucun appel synchrone sortant

Choix BD

MongoDB : événements JSON de formes variées, indicateurs calculés par pipeline d'agrégation.

MS7 — Interactions

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Appelle

Aucun appel sortant

MS7 consomme uniquement des événements

Appelé par · événements

De MS1, MS2
ReservationCreee, DemandeSangPubliee, DonEnregistre

De MS3, MS4
RendezVousReserve, PriseConfirmee, PriseOubliee

De MS6
VaccinEnregistre

Angular (Admin) via la Gateway
tableau de bord

En cas de panne

MS7 indisponible
messages gardés (files durables)

Au redémarrage
traitement, aucun événement perdu

Les autres services
continuent normalement

Aucun service ne dépend de MS7

Pourquoi — **Asynchrone uniquement : les statistiques n'ont pas besoin d'être en temps réel.**



Merci pour votre attention

MediConnect